CAPSTONE · PTY4614 · SECCIÓN 005D

Manual técnico y de despliegue

Instalación, configuración, migración y reversa del sistema

Dato	Valor
Proyecto	Sistema de Punto de Venta e Inventario — Emporio NaturalSur
Versión	v1.0 · 11-11-2026 · sistema con los módulos F1 a F7
Destinatario	Quien instale o mantenga el sistema después del proyecto
Base	Repositorio vroaseitz/sistema-inventario-productos: DESARROLLO.md, ADR 0001 a 0003, config/.env.example, build/pos.spec, database/migraciones, docs/09-plan-migracion

Estado del sistema al redactar este manual

Al 11-11-2026 el sistema funciona en una sola ventana con los siete módulos (F1 a F7) y está en la rama del Pull Request #19 hacia main. Las secciones marcadas «pendiente» corresponden a la migración desde Firebird y a la prueba en el equipo del local, que aún no se realizan.

1. Propósito

Este manual permite instalar el sistema en el computador del local, configurarlo, cargar los datos del sistema anterior, verificar que funciona y volver atrás si algo falla. Es uno de los mecanismos equivalentes comprometidos ante la docente guía al autorizar el despliegue sin contenedores (21- 08-2026, ADR 0003), y mitiga el riesgo de que las propietarias queden sin soporte técnico después del proyecto.

2. Arquitectura en una página

Componente	Descripción
Aplicación de escritorio	Python 3.11 con interfaz Tkinter, empaquetada como ejecutable con PyInstaller. No requiere Python instalado en el equipo del local.
Base local	SQLite (archivo pos_local.db) en modo WAL con synchronous=NORMAL y claves foráneas activas. Es la base con la que trabaja la aplicación; permite vender sin internet.
Base en la nube	Supabase (PostgreSQL gestionado), conexión por el pooler con TLS. Recibe copia de las operaciones.
Sincronización	Cola local de operaciones. Cada operación lleva un UUID y se publica con ON CONFLICT DO NOTHING, de modo que reintentar no duplica datos (ADR 0002).
Capas del código	src/pos/dominio (reglas), aplicacion (casos de uso y puertos), infraestructura (SQLite, Supabase, seguridad, impresora) e interfaz (Tkinter). La ventana única se arma en src/pos/interfaz/ventana_principal.py y se inicia con python -m pos.
Acceso	Usuarios locales con contraseña cifrada con bcrypt; la primera vez se crea la administradora. Perfiles Administradora y Vendedora (solo F1).

3. Requisitos

a. Equipo del local

Elemento	Requisito / situación verificada en terreno
Sistema operativo	Windows 8.1 de 64 bits. Es la razón de fijar Python 3.11: Python 3.12 ya no soporta Windows 8.1 (ADR 0001).

Procesador y memoria	Intel Core i5-3570 y 8 GB de RAM: suficientes para la aplicación.

Elemento	Requisito / situación verificada en terreno
Internet	Para sincronizar con Supabase. Sin conexión la venta continúa y la cola se publica al volver.
Lector de códigos	TxPOS USB, emula teclado. Debe estar configurado para enviar Enter al final de cada lectura (hallazgo B-01 del QA del 26-10).
Impresora y cajón	Xprinter 58 mm USB (ESC/POS); el cajón abre por comando de la impresora. Implementado con pywin32 (HU-VTA-06); pendiente de probar con el equipo real del local.
Energía	El equipo no tiene UPS. El modo WAL de SQLite protege la base ante un corte a mitad de una operación, pero se recomienda un UPS.

b. Equipo para desarrollar y construir

- Python 3.11 de 64 bits (por ejemplo 3.11.9) y Git.
- Para construir el ejecutable final: una máquina virtual con Windows 8.1 (o 7) y Python 3.11. Un ejecutable construido en Windows 10 u 11 no arranca en Windows 8.1 (ADR 0001 y 0003).
- Para la migración: copia del archivo PDVDATA.FDB y herramientas de Firebird 2.5 de 32 bits (sección 7).

4. Variables de entorno

La configuración vive en un archivo .env que se crea a partir de config/.env.example. El archivo .env está en .gitignore y nunca se sube al repositorio. La aplicación lo lee al iniciar (src/pos/infraestructura/config.py).

Variable	Uso	Valor por defecto
SUPABASE_DB_URL	Cadena de conexión a Supabase por el pooler (puerto 5432 en modo sesión). Contiene usuario y contraseña.	Sin valor: sin ella la aplicación trabaja solo en local
POS_SQLITE_PATH	Ruta del archivo de base local.	pos_local.db, junto al ejecutable

Seguridad de credenciales

La cadena de Supabase se entrega a quien instala por un canal privado, se escribe solo en el .env del equipo y no se copia en documentos, correos ni capturas.

5. Instalación

a. Entorno de desarrollo

1. Clonar el repositorio:git clone https://github.com/vroaseitz/sistema-inventario-productos.git
2. Crear y activar el entorno virtual: py -3.11 -m venv .venv y luego .venv\Scripts\activate
3. Instalar dependencias: pip install -r requirements-dev.txt (python-dotenv, psycopg2-binary, bcrypt, pywin32 en Windows, pytest y ruff).
4. Copiar config\.env.example a .env en la raíz y completar los valores.
5. Comprobar la calidad: ruff check . y pytest. Ambas deben terminar sin errores antes de construir.
6. Ejecutar: python -m pos (con PYTHONPATH=src). En el primer inicio se crea la base local y se pide la contraseña de la administradora.
7. Cargar las 20 categorías validadas: python scripts/cargar_categorias.py categorias.txt (una por línea).

Las pruebas que necesitan PostgreSQL se ejecutan solo si existe la variable POS_TEST_PG_URL; sin ella se omiten. GitHub Actions ejecuta la suite completa en cada envío.

b. Construcción del ejecutable

1. En la máquina virtual Windows 8.1, con el entorno virtual activo: pip install pyinstaller
2. Desde la raíz del repositorio: pyinstaller build/pos.spec
3. El resultado queda en la carpeta dist\pos-naturalsur\ (ejecutable pos-naturalsur.exe y sus bibliotecas).
4. Prueba de humo en la misma máquina virtual: abrir el ejecutable y registrar un producto de prueba.

c. Instalación en el computador del local

1. Copiar la carpeta dist\pos-naturalsur\ a C:\NaturalSur\ (ruta sugerida, fuera de Escritorio y Descargas).
2. 2. Crear el archivo .env en esa carpeta con las dos variables de la sección 4. 3. Crear un acceso directo a pos-naturalsur.exe en el escritorio.
3. 4. Primer inicio: la aplicación crea pos_local.db y su esquema automáticamente. 5. Verificar con la lista de la sección 8.

Pendiente: prueba de instalación en el equipo del local o en una máquina virtual idéntica con la versión de main.

6. Base de datos

a. Supabase

Los scripts de database/migraciones/ se aplican en orden en el editor SQL de Supabase (o con psql), una sola vez cada uno. Una migración ya aplicada nunca se modifica: los cambios se hacen con una nueva.

Script	Contenido
001_esquema_inicial.sql	Esquema inicial.
002_modelo_completo.sql	Modelo de datos completo: 20 tablas.
003_retirar_check_precio_bajo_costo.sql	Permite vender bajo el costo, necesario para liquidar productos por vencer (HU-INV-06).
004_sincronizacion.sql	Soporte de la sincronización sin conexión hacia Supabase (ADR 0002).

Script	Contenido
005_permisos_pos_app.sql	Rol pos_app con permisos acotados, que usa la aplicación.
006 a 008	Columnas para sincronizar lotes, pantallas, clientes, compras y auditoría.
009_cupo_credito.sql	Cupo de crédito por cliente.

La aplicación se conecta con el rol pos_app, que no es dueño de las tablas: las migraciones se aplican desde el editor SQL de Supabase con el rol postgres. Para las que lo requieren hay una copia lista para pegar (archivos .sql.txt). El script scripts/verificar_supabase.py comprueba en solo lectura que el esquema remoto tenga todas las columnas.

b. SQLite local

El esquema local se crea solo al iniciar (src/pos/infraestructura/sqlite/esquema.py). No requiere pasos manuales.

7. Migración de datos desde el sistema anterior

El sistema anterior (Abarrotes PDV 2.12) usa Firebird 2.x embebido en el archivo PDVDATA.FDB. La migración es un proceso único, separado de la aplicación, que no incluye drivers de Firebird (docs/09-plan-migracion).

1. Respaldar: copiar PDVDATA.FDB y trabajar siempre sobre la copia. El archivo del local es de solo lectura para el proyecto.
2. Exportar con las herramientas de Firebird 2.5 de 32 bits: gbak para un respaldo íntegro e isql para volcar cada tabla a CSV.
3. Depurar el catálogo según el protocolo acordado: de 4.109 fichas, 1.135 se vendieron desde el 02-03-2026.
4. Cargar los CSV limpios con el importador Python (pendiente) y verificar conteos contra el origen.
5. Ensayar el procedimiento completo en máquina virtual y registrar fecha, resultado e incidencias en docs/09-plan-migracion.

Error frecuente

«WindowsError [193]: %1 is not a valid Win32 application»: se intentó abrir la DLL de Firebird de 32 bits desde un Python de 64 bits. Usar las herramientas nativas de Firebird o un Python de 32 bits aislado solo para la migración.

8. Verificación posterior a la instalación

#	Prueba	Resultado esperado
1	Abrir la aplicación desde el acceso directo	Abre sin consola y sin errores
2	Leer un producto con el lector en el campo Código	Se agrega a la venta sin tocar el mouse
3	Abrir turno, vender en efectivo y cerrar turno	Arqueo con diferencia $0

#	Prueba	Resultado esperado
4	Cerrar y volver a abrir la aplicación	Las ventas y productos siguen ahí
5	Desconectar internet, vender, reconectar	La venta se publica en Supabase sin duplicarse
6	Revisar pos_local.db junto al ejecutable	Existe y crece con el uso

9. Respaldo

- Respaldar a diario el archivo pos_local.db. Hacerlo con la aplicación cerrada: en modo Wal hay datos que pueden estar todavía en los archivos pos_local.db-wal y pos_local.db-shm mientras está abierta.
- Guardar la copia fuera del computador del local (por ejemplo, un pendrive o la nube de las propietarias): hoy ese equipo es un punto único de falla.
- Supabase mantiene una segunda copia de las operaciones sincronizadas.

10. Problemas de reversa

a. Volver a una versión anterior de la aplicación

1. Cerrar la aplicación y respaldar pos_local.db.
2. Reemplazar la carpeta C:\NaturalSur\ por la versión anterior, conservando .env y pos_local.db.
3. Abrir y verificar con la sección 8.

b. Volver al sistema anterior

- El sistema anterior no se desinstala hasta terminar la marcha blanca.
- Si hay que volver, se restaura el respaldo gbak tomado antes del corte y se retoma Abarrotes PDV.
- Las ventas hechas en el sistema nuevo durante esos días se exportan desde pos_local.db para conciliarlas a mano.

- El plan de reversa se prueba con un restauración real en máquina virtual antes del corte, no solo se declara.

11. Problemas frecuentes

Síntoma	Causa y solución
El ejecutable no abre en Windows 8.1	Se construyó en Windows 10 u 11. Reconstruir en la máquina virtual Windows 8.1 (sección 5.2).
No conecta con Supabase	Revisar SUPABASE_DB_URL: debe usar el pooler (aws-0- <región>.pooler.supabase.com:5432), no la conexión directa.

Síntoma	Causa y solución
El lector escribe el código pero no lo agrega	Falta el sufijo Enter en el lector. Configurarlo con el código de programación de su manual.
El lector escribe símbolos raros	El lector usa otra distribución de teclado. Configurarlo en español o en modo numérico.
Error 193 al migrar	Mezcla de 32 y 64 bits con Firebird (sección 7).

12. Pendientes para cerrar este manual

- Aprobar el Pull Request #19 para que main tenga la versión con los siete módulos.
- Construir el ejecutable de esa versión en la máquina virtual Windows 8.1 y probarlo en el equipo del local, con la impresora y el cajón reales.
- Implementar y ensayar la migración desde Firebird (HU-MIG-01 a 03) y registrar el ensayo.
- Agregar capturas de cada paso de la instalación.